DRAFT · BUILT 2026-11-04

Writing the Weights: Hand-Programmed Policy Networks versus Imitation and Reinforcement Learning in an Island City-Builder

Dennis & Claude (Anthropic), working notes from the *New Shores* repository

ABSTRACT

We compare four ways to obtain the same small policy network, a multi-layer perceptron that reads 67 features of an economy and picks one of 26 actions in *New Shores*, a browser city-builder in the tradition of colony-building games: **imitation learning** of a rule-based policy (behavior cloning with DAgger), **reinforcement learning** from scratch (PPO), **weight-space programming** (WSP), where an AI coding agent writes every weight without an optimizer, and **WSP followed by PPO fine-tuning**. On 30 islands that no method saw during development, the imitation net never upgrades a single house. The hand-written net (28 hidden units; 165 of 2,658 weights non-zero) earns the game's final goal, the Imperial Charter, on 22 of 30 islands within 60 simulated minutes. PPO from scratch earns it on 27 when its actions are sampled, but on at most 14 when the net is played greedily, the way the game uses a net. Starting PPO from the hand-written weights gives the best policy on every measure: the Charter on 29 of 30 islands, 17% more progress than the best PPO run from scratch with the same compute, and, when only the hand-written weights and 36 spare units may change, the best greedy policy (24 of 30) while every hand-written unit keeps its meaning, so the changes RL made can be read unit by unit. Writing the weights also exposed six defects in the deterministic planner below the network that had capped every policy. Compute for reinforcement learning was cheap here ($22 for all runs); the advantage of hand-written weights lies in samples, about 34× fewer simulated decisions, which matters when samples are expensive and grows as token prices fall.

1 Introduction

A neural policy is usually something one trains. Weight-space programming inverts that: a programmer reads the network's weights as text, decides what each unit should compute, and writes the numbers directly [1]. The appeal is a network whose every unit has a stated purpose and can be checked, debugged and changed by hand. The open question is whether such a network can compete with one that was learned, and at what cost.

We study the question on a real, shipped artifact rather than a toy: the Advisor and Autopilot of *New Shores*, a city-builder written in plain JavaScript. The Advisor is an MLP that maps the island's state to the next action; a deterministic search then decides where to build and how to route roads. The game had shipped with a behavior-cloned net. Its users could watch the Autopilot grow hundreds of pioneers and never a single settler.

Our contributions: (i) a like-for-like comparison of imitation learning, reinforcement learning and weight-space programming on one network interface, one simulator and one seeded benchmark with held-out islands; (ii) a complete, unit-by-unit account of the hand-written network (Table 4, Figure 3), whose fine-tuned version now ships with the game, and an ablation of every unit (Table 5); (iii) hand-written weights as the starting point for reinforcement learning, with a readable record of what RL changed (Table 9); and (iv) a case study of what writing weights by hand reveals about the system around the network, and of what each approach costs.

2 Environment

2.1 The game

Each game starts on a procedurally generated archipelago of 96 × 96 tiles: a home island with a warehouse and 2 500 gold, plus two or three smaller islands. Each island has its own *fertilities*: sheep, grain and potatoes grow on some islands only, and spice never grows on the home island. Houses fill with residents while their needs are met and pay taxes. Population climbs four tiers. Pioneers need food and a marketplace. Settlers (unlocked at 24 pioneers) also need cloth and a chapel. Citizens (30 settlers) also need liquor and a tavern. Merchants (40 citizens) also need spice, which only a colony can grow. A full house upgrades automatically when the next tier is unlocked, its services cover it, the tier's signature good is in stock and the upgrade cost is affordable. The island *flourishes* at 60 citizens; housing 25 merchants earns the *Imperial Charter*, the end of the game. Events (pirates, fires, plague, storms, a trading ship with drifting prices) add noise.

2.2 The policy interface

Every 4 simulated seconds the policy sees a 67-dimensional feature vector: stocks and net rates of nine goods, gold (log scale), residents per tier, house fullness, the unlocked tier, counts of 18 building types, the share of houses lacking each service, a few binary flags inherited from the game's original rule-based policy (`foodLow`, `goldLow`, …), production-chain imbalances, and six features for colonies and spice. It outputs logits for 26 actions: wait, 20 building types, and five trades (sell surplus, buy tools, food, liquor or cloth). The game masks invalid actions (unaffordable, locked, or on a placement cooldown) and takes the arg-max. Placement and road routing are a deterministic search, outside the network. Feature and action lists are append-only, so older nets with fewer inputs or outputs keep working.

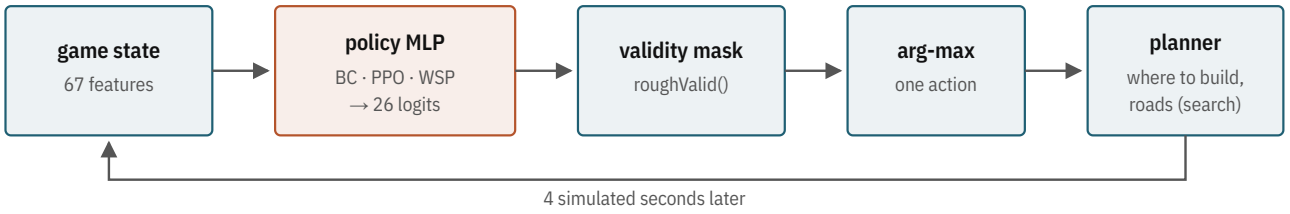


Figure 1. One decision. Only the second box differs between the methods compared here; mask, arg-max and planner are shared.

2.3 Evaluation protocol

The simulator runs headless in Node with seeded randomness, so every policy plays exactly the same games. We report *progress* = pioneers + 3·settlers + 6·citizens + 10·merchants, the residents per tier, and whether the Imperial Charter was earned, after 30 and 60 simulated minutes (450 and 900 decisions). A pure head-count would reward never upgrading, because upgrades cost gold and goods; the weights reflect that the game is won by climbing tiers. There are two island sets. The 14 **development islands** (seeds 11 … 606) were watched closely while the hand-written nets were developed, so results there favour WSP. The 30 **test islands** (seeds 2001–2030) were never used by any method. PPO trains on seeds ≥ 10 000 and selects checkpoints on seeds 901–907.

2.4 New Shores as an AI test bench

New Shores is more than the environment of this study. It serves its author as a test bench for AI methods, because one small, self-contained game exercises many of them at once, each against a concrete, visible result:

- **Image generation.** All building sprites, terrain textures and interface icons are generated locally with FLUX.1-schnell (4-bit, via mflux on Apple Silicon) and a post-processing pipeline that removes backgrounds, trims and downscales sprites and makes terrain textures tile seamlessly.
- **Animation.** The townspeople come from an AI-generated character sheet (Nano Banana) sliced into 61 cells, which form walk and work cycles (chopping, hammering, bending, carrying). Motion that diffusion models cannot draw consistently, such as bobbing, leaning and swinging, is animated procedurally by the engine.
- **Procedural audio.** A lookahead scheduler composes the soundtrack bar by bar and follows the game's mood (algorithmic, not learned).
- **Decision-making.** The Advisor and Autopilot policy is the subject of this paper: imitation learning, reinforcement learning and weight-space programming on one interface.
- **Code and text by language models.** In this study an AI coding agent fixed the game's defects, rewrote the building planner, added touch controls, wrote the hand-written networks, the training and benchmark scripts, and drafted this paper.

The headless, seeded simulator that makes the comparisons in this paper reproducible is part of the same bench: every method plays the same games and is judged by the same rules.

3 Methods

3.1 Imitation learning (BC)

The game's original advisor: an MLP (67 → 48 tanh → 26) trained by behavior cloning of a rule-based policy in the game's source (an if-cascade that grows houses before it ever considers a production chain, and so never builds a sheep farm): 28 games of 180 decisions with 18 % random exploration, followed by three DAgger rounds of 10 games each in which the net acts and the rule-based policy labels the states it visits. The cross-entropy loss is minimized by SGD with momentum; 'wait' labels are subsampled 1:4. Four candidates are trained and the best on three islands is kept. We retrained it on the final environment for this comparison.

3.2 Reinforcement learning (PPO)

Proximal policy optimization from random initialization, without any labels: policy MLP 67 → 64 tanh → 26 with a softmax over the valid actions only (the same mask as the game), a separate value MLP 67 → 64 → 1, reward = increase of progress per decision / 100, GAE with $\gamma = 0.995$ and $\lambda = 0.95$, clip $\varepsilon = 0.2$, entropy bonus 0.01, Adam at $3 \cdot 10^{-4}$, four epochs over minibatches of 512 per iteration. Training games are played in parallel worker processes on islands with seeds ≥ 10 000. Every 10 iterations the policy is scored on seven validation islands (seeds 901–907), both sampled and greedy, and the checkpoint with the best sampled score is kept as the run's result; the final weights are kept as well. We ran five configurations on a 256-core CPU server (Table 7c): 30-minute games with 14 games per iteration (*base*, 500 iterations), the same with a ten times smaller entropy bonus (*lowent*), with 56 games per iteration (*big*, 300 iterations), and 60-minute games with 14 (*h60*, 300 iterations) or 56 games per iteration (*h60big*, 200 iterations, 9.2 million decisions). The configuration was not tuned beyond these five runs.

3.3 Weight-space programming (WSP)

The weights are written by a programmer. Here the programmer was an AI coding agent (Claude) working in the repository with the user; it could read code, run the headless simulator and trace single games, but never ran an optimizer on the weights. The process went through three stages.

Stage 0, editing the trained net. First, gated units were appended to the BC net: each extra hidden unit $h = \tanh(g \cdot (\Sigma\, w_i x_i + b))$ is $\approx +1$ when a rule holds and ≈ -1 otherwise, and adds a priority p to one action through $W_2 = p/2$ and a bias shift of $p/2$, so the trained net is untouched while the rule is off. Edits were kept only if they raised progress on eight practice islands. Progress there rose from 363 to 736, but on six held-out islands only from 666 to 674. The edits fitted the practice islands; they did not generalize.

Stage 1, reading the trained net. Reading all 4 054 weights of the BC net at once was more useful. Its largest weights (|w| up to 28) sit on the binary flags of the rule-based policy it imitates: for instance a unit with `foodLow` +23.9 and `goldLow` −23.5 drives "buy food", and one with `woodLow` +8.6 drives "woodcutter". The net is that cascade written in weights. The input weights for cloth, potatoes and liquor are near zero in all 48 units, and the output columns for weaver, toolmaker, tavern and "buy tools" are near zero too: those branches of the imitated policy were almost never reached, so there was nothing to imitate. The `woodLow` flag is only 1 below four woodcutters, so the net could not even perceive a wood shortage in a large town.

Stage 2, WSP v2: needs → priorities. A new net written from scratch (the second hand-written design; a first one, with two hidden layers of switches and rules, was discarded). One hidden layer, the same shape as the original net. Each unit is a *need* read linearly from the features, graded rather than switched (food pressure rises smoothly with population and falls with stock), and conjunctions are folded into single units: each term is scaled to about ±2, and the bias subtracts 2 per extra term.

```
h_j = tanh( Σ_i W1[i][j]·x_i + b1[j] )            one need per unit
logit_a = −8 + Σ_j (p_ja / 2)·(h_j + 1)              p_ja: priority of need
action = arg-max over valid a  (ties and silence → wait)
```

3.4 Hand-written start, RL fine-tuning (WSP+PPO)

The two approaches can be combined: the hand-written net supplies what it knows about the game, and PPO tunes priorities and thresholds. The PPO policy (67 → 64 → 26) is initialized with WSP v2 in its first 28 hidden units. The remaining 36 units start with zero output weights, so before training the policy plays exactly like WSP v2, which we verified move for move on two islands. Three details make the warm start work. First, the hand-written priorities reach about 100, which makes a softmax over them practically deterministic; dividing all logits by a temperature of 10 keeps the arg-max, and so the greedy behaviour, unchanged while giving close alternatives a real probability. Second, the value net starts from random weights and its early advantage estimates are noise, so the first 20 iterations train only the value net and keep the policy fixed. Third, the hand-written units are saturated switches through which little gradient flows, so PPO mainly changes the output priorities and the spare units. We run two variants with the budget of the best PPO configuration (60-minute games, 56 per iteration, 200 iterations): **wsprl**, where every weight may change, and **wsprlsparse**, where only the 165 non-zero weights of WSP v2 and the spare units may change, so that the zeros, and with them the neuron-by-neuron account of Table 4, survive.

4 What writing the weights revealed

Every regression of a hand-written net could be traced to a specific unit and a specific game, which made a second kind of bug visible: defects below the network, in the deterministic planner.

None of them could have been learned around, because they removed the actions' effects altogether. Each was found by tracing why a unit that fired produced nothing.

1. **Walled-in base.** The planner preferred spots adjacent to the warehouse, which need no road. After a market, a chapel and six woodcutters, all eight tiles around the 2 × 2 warehouse were occupied and no road could ever connect to it again. Fix: keep the ring around every base free.
2. **Enclosed land.** Buildings placed wall to wall enclosed whole areas: on one island, 245 free tiles inside the building zone were unreachable by any road. Fix: a street grid, every fifth row and column from the warehouse kept free.
3. **Full zone.** Taverns (2 × 2) found no free spot in a full town. Fix: a depot at the zone's edge, which extends the zone.
4. **No room for a service.** When even that fails, the planner tears down up to four houses where a tavern or chapel covers the most unserved houses.
5. **Colonies used as overflow.** Fishers and houses filled the small colony island before any spice garden. Fix: on a colony, build only what the home island cannot grow.
6. **Lost pasture.** A spice garden lost two of its eight required grass tiles to later roads. Fix: farms are placed with a margin of four tiles.

The planner also routed roads outside the building zone, where placing them silently failed, and then built the unconnected building anyway; this, with ten other defects found in a separate code review, was fixed before any experiment reported here. Three trade actions (buy liquor, buy cloth) and the colony actions (Kontor, spice garden) were added to the interface because no policy could reach Merchants without them.

change	BC net	WSP v2
original planner (30 min)	493	768
+ free ring around bases	438	698
+ street grid, zone expansion, liquor import	624	1348

Table 1. Development log on the 14 development islands, mean progress after 30 minutes. The BC column is the net the game originally shipped with. Each row is the environment of its time, so rows do not compare across. WSP v2 has settlers on every island from the first row and is the first policy with citizens (107 per island on average in the last row).

WSP v2 milestone (60 min)	islands with citizens	with merchants	Charter
colony and spice actions, colony needs ≥ 2000 gold	14	4	4
colony from 1600 gold, liquor import gated by gold, colonies reserved for colony crops	12	7	7
cloth import from 5000 gold, depot need restored	12	8	8
houses cleared for taverns	14	7	7
early liquor import only from 2000 gold, pasture margin (final WSP v2)	**14**	**14**	**14**
rejected: houses shrink when a luxury is missing (stricter rules)	14	5	5

Table 2. Reaching the end of the game. Each row is one full benchmark run after a weight or planner change; mean of the final WSP v2: 34 pioneers, 212 settlers, 118 citizens, 107 merchants per island. The last row is a candidate rule change for the game (proposed in a code review) that was rejected because it made the game far harder to finish.

5 The hand-written network, neuron by neuron

Table 3 lists the networks. Table 4 is generated from the hand-written weights themselves: for each of the 28 hidden units, every non-zero input weight, the bias, every action it moves and by how much (the priority p, i.e. twice the output weight), and what it is for. All other weights are exactly zero.

net	layers	parameters	non-zero	density	how the weights were made
BC net	67 → 48 → 26	4,538	4,538	100.0 %	behavior cloning + DAgger (gradient descent)
WSP v2	67 → 28 → 26	2,658	165	6.2 %	hand-written: needs → priorities

Table 3. The networks compared. "Non-zero" counts weights and biases different from 0.

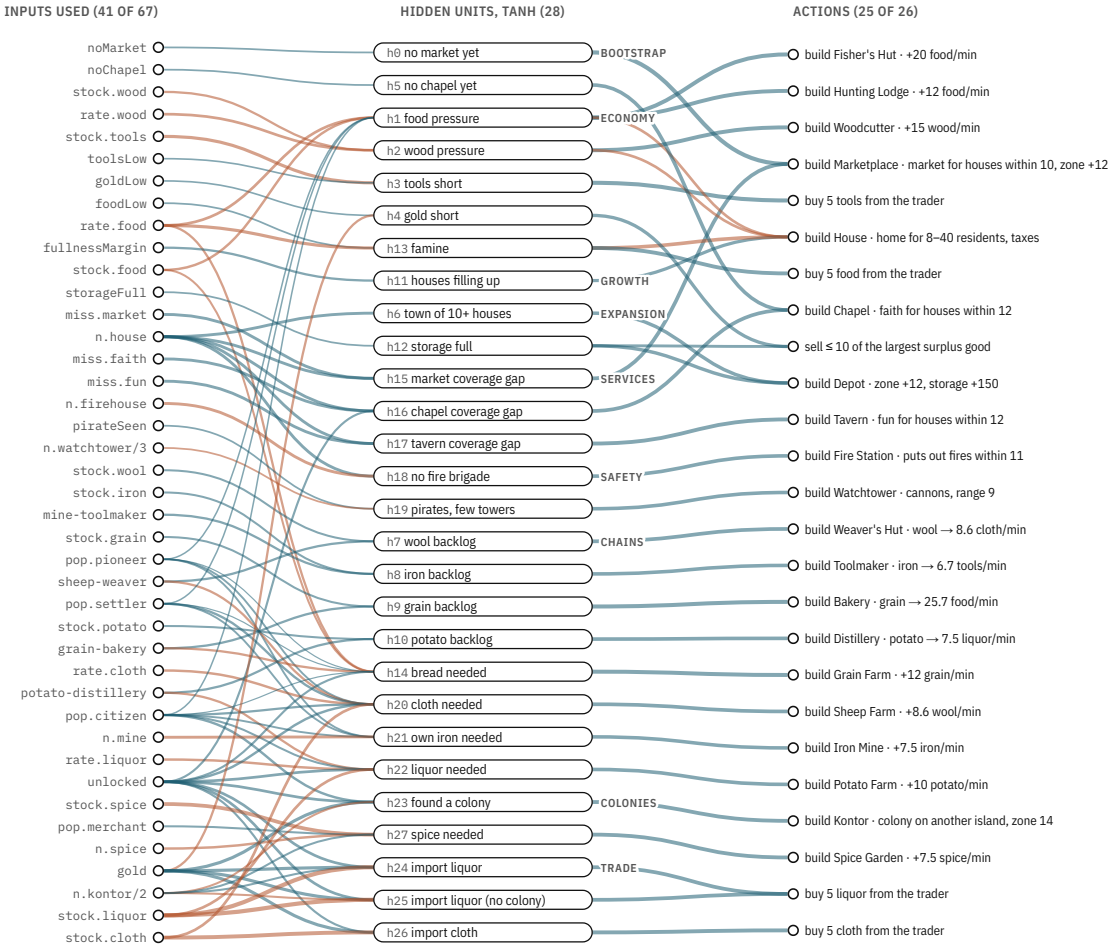


Figure 3. The hand-written network (WSP v2), drawn from its weights. Every line is one non-zero weight (112 in all): blue positive, rust negative, thickness grows with the logarithm of |w|. Left: the 41 features the network reads; the other inputs have no weight at all. Middle: the 28 hidden units, named after what they compute and grouped by role. Right: the 25 actions they drive, each with what it does: a building with its output per minute (inputs it consumes before the arrow) or service radius, or a trade with its amount. For a building, the planner picks the spot near a town centre and paves the road to it. Output lines carry the priority p of Table 4. Biases are not drawn; 'wait' has none and wins when nothing fires.

The units fall into eight groups. **Bootstrap** units (h0, h5) fire only on an empty island. **Economy** units (h1–h4, h13) are graded pressures on food, wood, tools and gold, plus a crisis unit that vetoes houses when the pantry is empty and falling; it is the only hard veto, because softer vetoes deadlocked islands with no fishing spot. **Chains** units keep producer/consumer pairs balanced (farms ≤ weavers, grain ≤ bakeries, …) and open a chain when its good is needed. **Services** units react to the share of houses without coverage. They were meant to be gated by "there are houses at all", because the service features read 1 on an empty island, but the gate is too weak: AND-ing linear terms is exact only when every term stays near ±2, and here a strongly true coverage term (+36) outvotes the false guard (−2). On an empty island these units therefore fire, and the market and chapel come first, which the bootstrap units ask for anyway. All results are measured with the net as written; the defect is a lesson for writing conjunctions (bounded terms, a 0/1 guard feature, or a second layer). **Safety** units add fire stations and watchtowers. **Expansion** units drive depots,

which extend the building zone. **Colonies** and **Trade** units carry the late game: found a colony when citizens and gold allow, plant spice, and spend only surplus gold on imports. Priorities order them: market 100, tools 90, bakery 86, fisher 85, grain 83, woodcutter 78, …, house 30.

#	neuron	pre-activation: tanh(Σ w·x + b)	priority added to action	role
h0	noMarket BOOTSTRAP	+6·noMarket −3	+100 **build Marketplace · market for houses within 10, zone +12**	Fires while the island has no marketplace at all. Pioneers will not move in without one, so it outbids everything.
h1	foodNeed ECONOMY	−20·stock.food −30·rate.food +4·pop.pioneer +4·pop.settler +4·pop.citizen +2	−15 **build House · home for 8–40 residents, taxes** +85 **build Fisher's Hut · +20 food/min** +75 **build Hunting Lodge · +12 food/min**	Graded food pressure: rises with population, falls with food in store and with a positive food balance. Calls for fishers (first) and hunters, and dampens new houses.
h2	woodNeed ECONOMY	−15·stock.wood −30·rate.wood +3	−15 **build House · home for 8–40 residents, taxes** +78 **build Woodcutter · +15 wood/min**	Graded wood pressure from stock and rate. Has no cap on woodcutters, unlike the game's original rule-based policy, whose cap of 4 starved large towns of planks.
h3	toolsNeed ECONOMY	−60·stock.tools +5·toolsLow +2.4	+90 **buy 5 tools from the trader**	Fewer than ~12 tools in store, strongly below 3: buy tools. Upgrades and most buildings consume tools.
h4	goldNeed ECONOMY	−15·gold +5·goldLow +9	+35 **sell ≤ 10 of the largest surplus good**	Treasury under ~250 gold: sell surplus goods.
h5	noChapel BOOTSTRAP	+6·noChapel −3	+50 **build Chapel · faith for houses within 12**	No chapel on the island yet.
h6	town10 EXPANSION	+40·n.house −38	+42 **build Depot · zone +12, storage +150**	10 or more houses. Drives a depot: a second road anchor that extends the building zone, so the town spreads instead of walling itself in.
h7	weaverLT CHAINS	+10·stock.wool +15·sheep-weaver −1	+58 **build Weaver's Hut · wool → 8.6 cloth/min**	More sheep farms than weavers, or wool piling up: build a weaver.
h8	toolmakerLT CHAINS	+10·stock.iron +15·mine-toolmaker −1	+47 **build Toolmaker · iron → 6.7 tools/min**	More iron mines than toolmakers, or iron piling up: build a toolmaker.
h9	bakeryLT CHAINS	+10·stock.grain +15·grain-bakery −1	+86 **build Bakery · grain → 25.7 food/min**	More grain farms than bakeries, or grain piling up: build a bakery.
h10	distilleryLT CHAINS	+10·stock.potato +15·potato-distillery −1	+53 **build Distillery · potato → 7.5 liquor/min**	More potato farms than distilleries, or potatoes piling up: build a distillery.
h11	growth GROWTH	+15·fullnessMargin +1.5	+30 **build House · home for 8–40 residents, taxes**	Houses are more than ~45 % full (or there are none yet): build another house. The default activity once nothing is urgent.

#	neuron	pre-activation: tanh(Σ w·x + b)	priority added to action	role
h12	`storageFull` **EXPANSION**	`+6·storageFull −3`	`+40` **build Depot · zone +12, storage +150** `+20` **sell ≤ 10 of the largest surplus good**	Some good is at the storage cap: build a depot, or sell.
h13	`foodCrisis` **ECONOMY**	`−60·rate.food +6·foodLow −5`	`−50` **build House · home for 8–40 residents, taxes** `+70` **buy 5 food from the trader**	Fewer than 25 food and the balance is falling: emergency food purchase and a hard veto on new houses.
h14	`grainTrigger` **CHAINS**	`−10·stock.food −15·rate.food +2·pop.pioneer +2·pop.settler +2·pop.citizen +12·unlocked −15·grain-bakery −7`	`+83` **build Grain Farm · +12 grain/min**	Food pressure once Settlers are unlocked and farms ≤ bakeries: build a grain farm (bread scales beyond fishing).
h15	`marketTrigger` **SERVICES**	`+40·n.house +40·miss.market −8`	`+60` **build Marketplace · market for houses within 10, zone +12**	More than ~10 % of existing houses lack a marketplace.
h16	`faithTrigger` **SERVICES**	`+12·unlocked +40·n.house +40·miss.faith −18`	`+55` **build Chapel · faith for houses within 12**	Settlers unlocked and >15 % of houses lack chapel coverage.
h17	`funTrigger` **SERVICES**	`+40·n.house +40·miss.fun −10`	`+70` **build Tavern · fun for houses within 12**	>15 % of houses lack a tavern. Placed above the cloth chain: a Citizen upgrade needs tavern coverage.
h18	`fireTrigger` **SAFETY**	`+40·n.house −40·n.firehouse −38`	`+50` **build Fire Station · puts out fires within 11**	10+ houses and no fire station.
h19	`towerTrigger` **SAFETY**	`+6·pirateSeen −6·n.watchtower/3 −3`	`+48` **build Watchtower · cannons, range 9**	Pirates have been sighted and there are fewer than 2 watchtowers.
h20	`sheepTrigger` **CHAINS**	`−30·stock.cloth −20·rate.cloth +8·pop.pioneer +18·pop.settler +14·pop.citizen +20·unlocked −15·sheep-weaver −18.4`	`+55` **build Sheep Farm · +8.6 wool/min**	Cloth demand (Settlers unlocked, settlers and citizens eating cloth, little in store) with farms ≤ weavers and a town of ~60+ people that can carry the investment.
h21	`mineTrigger` **CHAINS**	`+8·pop.pioneer +8·pop.settler +8·pop.citizen −40·n.mine −3.5`	`+45` **build Iron Mine · +7.5 iron/min**	No iron mine yet and a town big enough to invest.
h22	`potatoTrigger` **CHAINS**	`−30·stock.liquor −20·rate.liquor +10·pop.citizen +30·unlocked −15·potato-distillery −29.5`	`+50` **build Potato Farm · +10 potato/min**	Citizens unlocked, liquor short, potato farms ≤ distilleries.
h23	`colonyTrigger` **COLONIES**	`+60·gold +20·pop.citizen +30·unlocked −10·n.kontor/2 −83`	`+66` **build Kontor · colony on another island, zone 14**	Citizens unlocked, 20+ citizens, no Kontor yet and ≥ ~1600 gold: found a colony. The only road to spice, Merchants and the Imperial Charter.

#	neuron	pre-activation: tanh(Σ w·x + b)	priority added to action	role
h24	`liquorImport` **TRADE**	`−300·stock.liquor +60·gold +30·unlocked +10·n.kontor/2 −78`	`+60` **buy 5 liquor from the trader**	Liquor almost gone, a colony exists and ≥ ~1000 gold: import liquor (5 units buy several upgrades).
h25	`liquorImportEarly` **TRADE**	`−300·stock.liquor +60·gold +30·unlocked −10·n.kontor/2 −78.5`	`+60` **buy 5 liquor from the trader**	The same before a colony exists, but only from ~2000 gold, so imports never eat the Kontor's 1500.
h26	`clothImport` **TRADE**	`−200·stock.cloth +60·gold +12·unlocked −59.5`	`+72` **buy 5 cloth from the trader**	Cloth almost gone and ≥ ~5000 idle gold: import cloth. Converts surplus gold into upgrades.
h27	`spiceTrigger` **COLONIES**	`−150·stock.spice +10·pop.merchant +10·n.kontor/2 −20·n.spice −1`	`+64` **build Spice Garden · +7.5 spice/min**	A colony exists, little spice in store, few gardens for the number of merchants: build a spice garden.

Table 4. All 28 hidden units of the hand-written network (dev/handwritten-net-v2.js; regenerate with `node dev/explain-net.js`). Feature scales: stocks are shares of storage (300 plus depots), rates are units per minute / 20, populations are residents / 100, building counts / 10, gold is $\log_{10}(1 + \text{gold}) / 4$.

5.1 Ablation: what each unit is worth

To check that every unit does what its name says, we switched off one unit at a time (no inputs, bias −100, so it never fires) and replayed the 14 development islands for 60 minutes (Table 5). Four units are indispensable: without *houses filling up* (h11) no house is ever built, without *wood pressure* (h2) the economy never starts, without *food pressure* (h1) towns starve, and without *tavern coverage gap* (h17) no island reaches Citizens. The late game rests on a chain of units, each of which removes the Charter on every island when missing: *found a colony* (h23), *spice needed* (h27) and the tavern unit. Several units cost little or no progress but still lose Charters when missing (fire brigade, storage, gold), which is the insurance they were written for. One unit is dead weight: *import cloth* (h26) changes progress by +1 and costs one Charter, within noise. It survived development because it was added together with another change that did help (Table 2, row 3), and is a candidate for removal.

#	unit switched off	loss of progress	Δ	progress	citizens on	Charter
	none (WSP v2 as written)		**—**	**2,445**	**14/14**	**14/14**
h11	houses filling up		−2,445	0	0/14	0/14
h2	wood pressure		−2,339	106	0/14	0/14
h1	food pressure		−1,917	528	6/14	1/14
h17	tavern coverage gap		−1,228	1,217	0/14	0/14
h7	wool backlog		−1,005	1,440	14/14	6/14
h3	tools short		−908	1,537	7/14	7/14
h20	cloth needed		−739	1,706	14/14	5/14
h24	import liquor		−674	1,771	11/14	5/14
h23	found a colony		−590	1,855	14/14	0/14
h27	spice needed		−487	1,958	14/14	0/14
h6	town of 10+ houses		−393	2,052	13/14	10/14
h18	no fire brigade		−316	2,129	13/14	13/14
h21	own iron needed		−276	2,168	13/14	12/14
h25	import liquor (no colony)		−256	2,188	12/14	12/14
h12	storage full		−227	2,218	14/14	11/14
h15	market coverage gap		−193	2,251	13/14	13/14
h5	no chapel yet		−180	2,264	14/14	12/14
h16	chapel coverage gap		−141	2,304	14/14	13/14
h8	iron backlog		−129	2,315	13/14	13/14
h10	potato backlog		−78	2,367	14/14	14/14
h19	pirates, few towers		−71	2,374	14/14	14/14
h13	famine		−57	2,388	14/14	13/14
h0	no market yet		−54	2,391	14/14	14/14
h14	bread needed		−47	2,398	14/14	13/14
h22	liquor needed		−37	2,408	14/14	13/14
h9	grain backlog		−25	2,420	14/14	13/14
h26	import cloth		+1	2,445	13/14	13/14
h4	gold short		+69	2,513	14/14	11/14

Table 5. Ablation of each hidden unit of WSP v2, 14 development islands, 60 minutes, sorted by loss of progress. Bars scale with the loss; "citizens on" counts islands that reach the Citizen tier.

6 Results

Table 7 is the main result. Four findings stand out. **Imitation fails:** the BC net never upgrades a house, on any island. **The hand-written net works but overfits:** WSP v2 earns the Charter on 22 of 30 unseen islands, against 14 of 14 on the development islands it was written for (Table 7b). **PPO from scratch learns more than the programmer wrote down, but only as a stochastic policy:** the best run, sampled, earns the Charter on 27 of 30 islands and reaches it fastest among the single methods, while the same net played greedily earns it on 13 or 14. **The combination is best:** PPO started from the hand-written weights earns the Charter on 29 of 30 islands, in a median of 29 minutes, with the highest progress of all policies; its sparse variant is also the best greedy policy (24 of 30), the form in which the game can use it. A policy that picks uniformly among the valid actions earns the Charter on 14 of 30 islands. The planner, after the fixes of section 4, carries much of the game; the imitation net, whose learned order (houses before any production chain) is worse than no order at all, earns it on none.

policy	progress 30 min	progress 60 min	P / S / C / M at 60 min	Charter	median min to Charter
uniform random (valid actions)	929	1,409	405 / 130 / 68 / 21	14/30	45
BC net (imitation)	918	933	933 / 0 / 0 / 0	0/30	–
WSP v2 (hand-written)	1,327	2,191	40 / 221 / 117 / 79	22/30	37
PPO h60big (60-min, 4× batch), best checkpoint, sampled	2,372	2,693	614 / 210 / 106 / 81	27/30	33
PPO h60big (60-min, 4× batch), best checkpoint, greedy	1,952	2,187	672 / 234 / 84 / 31	13/30	40
PPO h60big (60-min, 4× batch), final checkpoint, sampled	2,355	2,624	690 / 202 / 97 / 75	28/30	33
PPO h60big (60-min, 4× batch), final checkpoint, greedy	2,144	2,370	712 / 231 / 99 / 37	14/30	37
WSP+PPO wsprl (WSP v2 init, all weights), best checkpoint, sampled	2,691	**3,153**	379 / 274 / 177 / 89	**29**/30	29
WSP+PPO wsprl (WSP v2 init, all weights), best checkpoint, greedy	2,482	2,631	518 / 330 / 124 / 38	11/30	29
WSP+PPO wsprl (WSP v2 init, all weights), final checkpoint, sampled	2,506	2,888	464 / 271 / 139 / 78	26/30	32
WSP+PPO wsprl (WSP v2 init, all weights), final checkpoint, greedy	2,517	2,656	530 / 268 / 137 / 50	17/30	30
WSP+PPO wsprlsparse (WSP v2 init, sparse), best checkpoint, sampled	2,564	3,082	419 / 271 / 134 / 104	28/30	30

policy	progress 30 min	progress 60 min	P / S / C / M at 60 min	Charter	median min to Charter
WSP+PPO wsprlsparse (WSP v2 init, sparse), best checkpoint, greedy	2,274	2,599	484 / 299 / 127 / 46	12/30	33
WSP+PPO wsprlsparse (WSP v2 init, sparse), final checkpoint, sampled	2,478	3,137	416 / 307 / 160 / 84	26/30	33
WSP+PPO wsprlsparse (WSP v2 init, sparse), final checkpoint, greedy	2,493	2,942	460 / 276 / 123 / 92	24/30	32

Table 7. Main result: 30 test islands never used by any method, 60-minute games with a snapshot at 30 minutes. P / S / C / M: pioneers, settlers, citizens, merchants per island. *Sampled* plays a PPO net as it was trained, drawing actions from its softmax; *greedy* plays it as the game plays every net, taking the arg-max (greyed). *Best checkpoint*: the one with the best sampled score on the validation islands, chosen before testing; *final*: the weights after the last iteration, shown because the greedy policies kept improving after the best sampled checkpoint. Best values in bold.

policy	progress 30 min	progress 60 min	P / S / C / M at 60 min	Charter	median min to Charter
uniform random (valid actions)	865	1,357	373 / 153 / 60 / 17	5/14	51
BC net (imitation)	808	833	833 / 0 / 0 / 0	0/14	–
WSP v2 (hand-written)	1,491	2,445	34 / 212 / 118 / 107	**14**/14	37
PPO h60big (60-min, 4× batch), best checkpoint, sampled	2,367	2,627	584 / 252 / 95 / 72	12/14	35
PPO h60big (60-min, 4× batch), best checkpoint, greedy	1,965	2,054	669 / 263 / 68 / 19	5/14	42
PPO h60big (60-min, 4× batch), final checkpoint, sampled	2,357	2,672	630 / 235 / 113 / 66	11/14	34
PPO h60big (60-min, 4× batch), final checkpoint, greedy	2,149	2,199	715 / 234 / 90 / 24	7/14	45
WSP+PPO wsprl (WSP v2 init, all weights), best checkpoint, sampled	2,615	3,097	366 / 282 / 148 / 100	**14**/14	29
WSP+PPO wsprl (WSP v2 init, all weights), best checkpoint, greedy	2,496	2,735	500 / 320 / 168 / 26	6/14	31
WSP+PPO wsprl (WSP v2 init, all weights), final checkpoint, sampled	2,524	3,006	377 / 282 / 146 / 91	12/14	33
WSP+PPO wsprl (WSP v2 init, all weights), final checkpoint, greedy	2,691	2,771	504 / 257 / 159 / 54	8/14	31
WSP+PPO wsprlsparse (WSP v2 init, sparse), best checkpoint, sampled	2,652	**3,207**	398 / 305 / 170 / 87	13/14	30

policy	progress 30 min	progress 60 min	P / S / C / M at 60 min	Charter	median min to Charter
WSP+PPO wsprlsparse (WSP v2 init, sparse), best checkpoint, greedy	2,304	2,549	471 / 328 / 122 / 36	6/14	37
WSP+PPO wsprlsparse (WSP v2 init, sparse), final checkpoint, sampled	2,450	3,124	417 / 270 / 185 / 79	**14**/14	32
WSP+PPO wsprlsparse (WSP v2 init, sparse), final checkpoint, greedy	2,476	2,872	457 / 301 / 141 / 66	11/14	35

Table 7b. The same on the 14 development islands, on which the hand-written net was developed and which therefore favour it.

policy	progress 30 min	progress 60 min	P / S / C / M at 60 min	Charter	median min to Charter
PPO base (30-min games), best checkpoint, sampled	2,364	2,076	560 / 252 / 127 / 0	0/30	–
PPO base (30-min games), best checkpoint, greedy	1,924	1,879	639 / 360 / 27 / 0	0/30	–
PPO h60 (60-min games), best checkpoint, sampled	2,239	2,554	634 / 199 / 105 / 69	27/30	37
PPO h60 (60-min games), best checkpoint, greedy	1,732	1,816	713 / 230 / 69 / 0	0/30	–
PPO big (30-min, 4× batch), best checkpoint, sampled	2,517	2,235	631 / 233 / 148 / 2	1/30	45
PPO big (30-min, 4× batch), best checkpoint, greedy	2,190	2,203	676 / 265 / 122 / 0	0/30	–
PPO lowent (30-min, low entropy), best checkpoint, sampled	2,140	1,897	666 / 411 / 0 / 0	0/30	–
PPO lowent (30-min, low entropy), best checkpoint, greedy	1,978	1,816	723 / 364 / 0 / 0	0/30	–
PPO h60big (60-min, 4× batch), best checkpoint, sampled	2,372	**2,693**	614 / 210 / 106 / 81	27/30	33
PPO h60big (60-min, 4× batch), best checkpoint, greedy	1,952	2,187	672 / 234 / 84 / 31	13/30	40
PPO h60 (60-min games), final checkpoint, sampled	2,234	2,495	648 / 206 / 91 / 69	26/30	39
PPO h60 (60-min games), final	1,840	1,839	763 / 264 / 44 / 2	1/30	55

policy	progress 30 min	progress 60 min	P / S / C / M at 60 min	Charter	median min to Charter
checkpoint, greedy					
PPO h60big (60-min, 4× batch), final checkpoint, sampled	2,355	2,624	690 / 202 / 97 / 75	**28**/30	33
PPO h60big (60-min, 4× batch), final checkpoint, greedy	2,144	2,370	712 / 231 / 99 / 37	14/30	37

Table 7c. All PPO-from-scratch configurations on the 30 test islands. Only the runs trained on 60-minute games learn to found colonies and reach Merchants; the 30-minute runs are strongest after 30 minutes and never get there.

6.1 Accuracy is the wrong yardstick

The original advisor was described by its accuracy: how often it chose the same action as the rule-based policy it imitates. Table 6 shows why that number misleads here. The retrained imitation net agrees with that policy two thirds of the time and keeps 92 % of its houses fully supplied, because it only ever builds pioneer houses, whose needs are food and a market. WSP v2 disagrees with it three times out of four, and still keeps 78 % of its houses fully supplied although most of them are settler, citizen or merchant houses that also need cloth, liquor, spice, a chapel and a tavern.

policy	accuracy (agrees with imitated policy)	houses fully supplied
Old advisor (BC, as shipped)	47.6 %	70.4 %
BC net (retrained)	66.4 %	92.0 %
WSP v2	24.7 %	78.5 %

Table 6. Accuracy (share of decisions identical to the imitated rule-based policy's in the same state), and share of houses with every need met (sampled every minute), 14 development islands, 30 minutes. "Old advisor" is the net the game shipped with before this study (61 inputs, 22 actions).

6.2 Reinforcement learning: sampled and greedy

A PPO policy is a distribution over actions, and what it learns is good as a distribution. Played greedily, by always taking its most probable action, it is consistently worse (Table 7) and was, for part of training, almost useless: in every one of the five runs from scratch the greedy validation score first rose and then collapsed to between 0 and 24 for a stretch of iterations, before recovering as the policy sharpened (Figure 2). The two WSP-initialized runs never collapsed; their greedy score stayed above 1 670 throughout. The warm start keeps the policy's arg-max meaningful while RL explores around it. Two practical lessons follow. The episode must be long enough to contain the goal: only runs trained on 60-minute games ever found a colony. And checkpoints should be selected in the form in which the policy will be used: selected by their sampled score, the WSP-ini-

tialized runs' best checkpoints earn the Charter greedily on 11 and 12 of 30 islands, their final weights on 17 and 24.

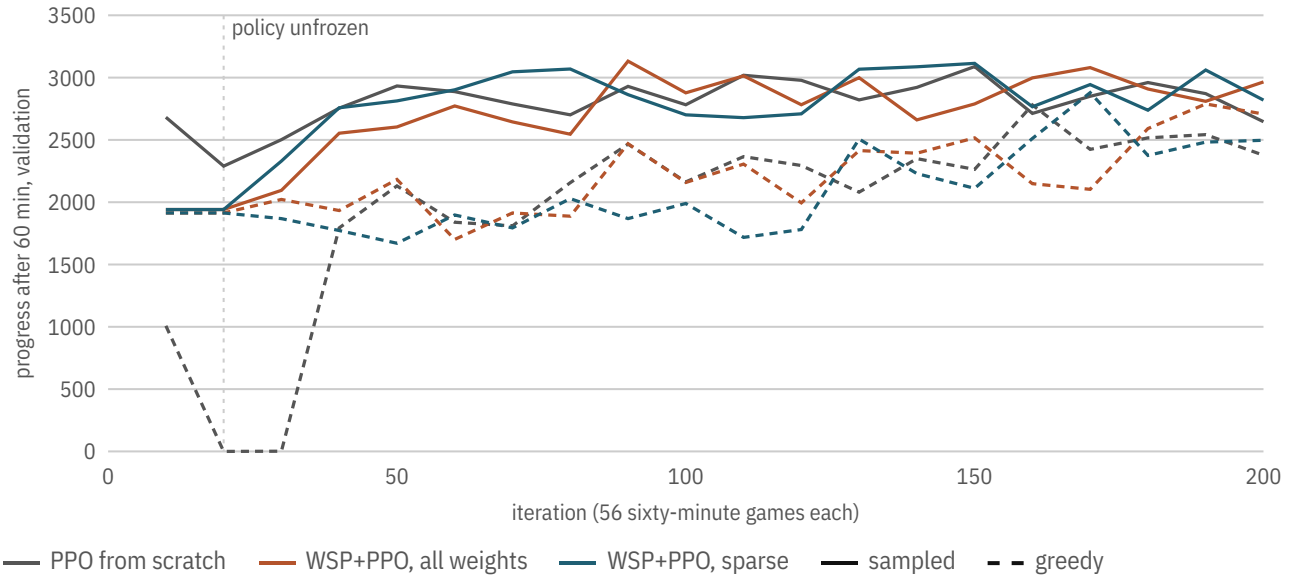


Figure 2. Validation progress after 60 minutes, every 10 iterations, for the best PPO configuration from scratch and the two WSP-initialized runs with the same budget. Solid: sampled actions; dashed: greedy. The WSP-initialized runs train only their value net for the first 20 iterations. The hand-written net alone reaches about 2 450 on the development islands.

6.3 What reinforcement learning changed in the hand-written net

Because the sparse variant may only change the hand-written net's non-zero weights and its 36 new units, its result can be compared with the original unit by unit (Table 9). The input weights of every hand-written unit moved by at most a few percent, so each unit still detects what its name says. What PPO changed are the priorities, and its changes are legible. It almost switched off the import of liquor before a colony exists (priority 60 → 19), the interaction that had cost the hand-written development most: liquor bought early ate the gold the colony needs. It lowered the cloth import (72 → 62), which the ablation had found to be dead weight. It raised the toolmaker (47 → 60), the distillery (53 → 61) and the colony (66 → 69), and lowered the food producers, the woodcutter and the tavern by a few points. The spare units carry a substantial share of the decisions and are not explained; they are where RL added rules the programmer did not write. Without the sparsity mask, the same training rewired the hand-written units themselves: their input weights moved by up to 170 %, and every unit gained connections to many actions, so the account of Table 4 no longer applies to that net. Its results are mixed against the sparse variant: better when sampled at the best checkpoint (3 153 against 3 082), worse when played greedily at the end (2 656 against 2 942, 17 against 24 Charters). Keeping the program readable cost nothing measurable.

#	hand-written unit	action	priority written	after PPO	Δ	input drift
h25	import liquor (no colony)	buy 5 liquor from the trader	60	19	−41	0.1 %
h8	iron backlog	build Toolmaker · iron → 6.7 tools/min	47	60	+13	0.6 %
h26	import cloth	buy 5 cloth from the trader	72	62	−10	0.8 %
h10	potato backlog	build Distillery · potato → 7.5 liquor/min	53	61	+8	1.9 %
h17	tavern coverage gap	build Tavern · fun for houses within 12	70	65	−5	0.0 %
h19	pirates, few towers	build Watchtower · cannons, range 9	48	43	−5	4.3 %
h2	wood pressure	build Woodcutter · +15 wood/min	78	73	−5	0.2 %
h14	bread needed	build Grain Farm · +12 grain/min	83	79	−4	0.3 %
h4	gold short	sell ≤ 10 of the largest surplus good	35	39	+4	3.2 %
h12	storage full	sell ≤ 10 of the largest surplus good	20	24	+4	2.2 %
h1	food pressure	build Hunting Lodge · +12 food/min	75	71	−4	0.6 %
h23	found a colony	build Kontor · colony on another island, zone 14	66	69	+3	1.2 %
h9	grain backlog	build Bakery · grain → 25.7 food/min	86	83	−3	1.4 %
h1	food pressure	build Fisher's Hut · +20 food/min	85	82	−3	0.6 %
h24	import liquor	buy 5 liquor from the trader	60	57	−3	0.1 %
h18	no fire brigade	build Fire Station · puts out fires within 11	50	47	−3	0.5 %
h20	cloth needed	build Sheep Farm · +8.6 wool/min	55	52	−3	3.0 %
h15	market coverage gap	build Marketplace · market for houses within 10, zone +12	60	57	−3	0.1 %
h0	no market yet	build Marketplace · market for houses within 10, zone +12	100	103	+3	2.5 %
h7	wool backlog	build Weaver's Hut · wool → 8.6 cloth/min	58	55	−3	1.0 %
h16	chapel coverage gap	build Chapel · faith for houses within 12	55	53	−2	0.0 %
h1	food pressure	build House · home for 8–40 residents, taxes	-15	-13	+2	0.6 %
h5	no chapel yet	build Chapel · faith for houses within 12	50	52	+2	1.4 %
h21	own iron needed	build Iron Mine · +7.5 iron/min	45	47	+2	2.3 %
h13	famine	buy 5 food from the trader	70	72	+2	0.8 %
h22	liquor needed	build Potato Farm · +10 potato/min	50	49	−1	0.9 %

#	hand-written unit	action	priority written	after PPO	Δ	input drift
h11	houses filling up	build House · home for 8–40 residents, taxes	30	31	+1	5.2 %
h27	spice needed	build Spice Garden · +7.5 spice/min	64	63	−1	0.2 %
h3	tools short	buy 5 tools from the trader	90	91	+1	2.0 %
h6	town of 10+ houses	build Depot · zone +12, storage +150	42	43	+1	0.1 %
h2	wood pressure	build House · home for 8–40 residents, taxes	-15	-15	+0	0.2 %
h13	famine	build House · home for 8–40 residents, taxes	-50	-50	−0	0.8 %
h12	storage full	build Depot · zone +12, storage +150	40	40	−0	2.2 %

36 spare units added by PPO: their summed priorities range from 63 to 155 (median 82). Input drift = Σ|Δw| / Σ|w| over the unit's input weights.

Table 9. Priorities of the hand-written units before and after PPO fine-tuning (sparse variant, final weights), sorted by the size of the change. Priorities after PPO are the output weights × 2 × the temperature of 10 used at initialization, so both columns are on the same scale.

6.4 Cost

A common argument for writing weights with a language model is cost: tokens are cheap, training is expensive. For this study the argument does not hold, and the reason is instructive. Table 8 lists what each method consumed. The simulator is fast (6.6 ms per decision on a laptop core; 99.3 % of that time is the game, 0.7 % the network) and the network is tiny, so PPO needs no GPU, and all 7 PPO runs of this study together cost $22 at cloud prices for CPU time [5]; the best single run cost $5.01. The tokens spent on writing the weights are of the same order. We have no billing export for the study and estimate them from the number of requests and their context lengths at the list prices of the models used [6]: about $15–30 for the development of WSP v2, and about $30–60 for the whole study including the game fixes and this paper.

Where hand-written weights do save is in *samples*. WSP v2 was developed with roughly 0.3 million simulated decisions, mostly benchmark runs; the best PPO run used 10.1 M, some 34 times more. Here a simulated decision costs a fraction of a millisecond of CPU time, so the difference is worth cents. It decides the comparison whenever a sample is expensive: a slow physical simulator, real hardware, or a person in the loop.

The two costs also move differently over time (Figure 4). Prices for a fixed level of LLM capability have fallen by between 9× and 900× per year depending on the task, with a median of about 50× [7]; other estimates put it near 10× per year [8] or 5–10× for frontier models [9]. List prices of the most capable models fall more slowly: Anthropic's top tier went from $75 per million output tokens (Claude 3 Opus, March 2024) to $25 (Claude Opus 4.5, November 2025), roughly 2× per year [6]. CPU time has changed little, and we assume it flat. If the work done here by a frontier

model can be done in a year by a cheaper model of today's capability, the estimated token cost of writing such a network drops below the CPU cost of the best PPO run within 5–8 months; if it must wait for frontier list prices to fall, it takes about 2.2 years.

method	simulated decisions	CPU	CPU cost	LLM tokens	note
BC net (dev/train.js)	≈ 0.05 M	≈ 0.2 core-h	$0.01	—	imitates a rule-based policy written by hand beforehand
WSP v2 (this study)	≈ 0.30 M	≈ 2 core-h	$0.10	≈ $15–30 (est.)	an AI agent writing weights; a person steering
PPO base	3.1 M	25 core-h	$1.27	—	500 iterations, 99 min wall time
PPO h60	3.8 M	25 core-h	$1.28	—	300 iterations, 100 min wall time
PPO big	7.6 M	76 core-h	$3.90	—	300 iterations, 80 min wall time
PPO lowent	3.1 M	25 core-h	$1.27	—	500 iterations, 99 min wall time
PPO h60big	10.1 M	98 core-h	$5.01	—	200 iterations, 103 min wall time
PPO wsprl	10.1 M	88 core-h	$4.51	—	200 iterations, 93 min wall time
PPO wsprlsparse	10.1 M	87 core-h	$4.48	—	200 iterations, 92 min wall time
all 7 PPO runs	47.9 M	424 core-h	$22	—	every configuration tried, incl. the WSP-initialized runs

Table 8. Resources used per method. PPO rows are computed from the training logs: wall time × (workers + 1) cores, at $0.0513 per vCPU-hour (AWS c7a, on-demand) [5]. Token costs are estimates at list prices [6]; the person who steered the work is not priced. BC and WSP CPU times are approximate.

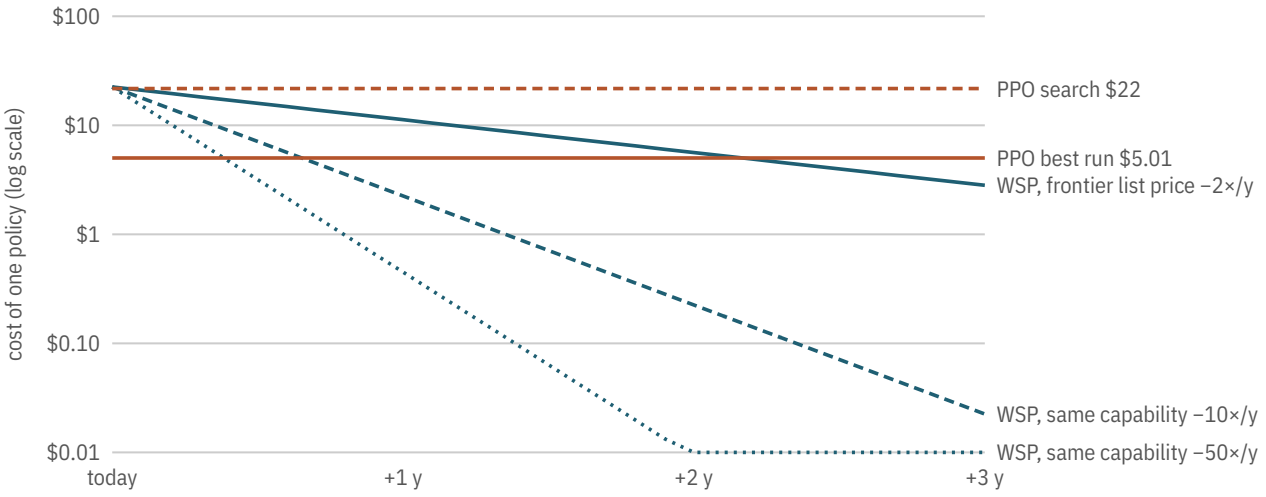


Figure 4. Projected cost of producing one policy. Blue: the estimated token cost of WSP v2 (midpoint of the estimate) falling at the frontier list-price rate (≈ 2×/year [6]) and at the rates reported for a fixed capability (10× [8] and 50× per year [7]). Rust: CPU cost of the best PPO run and of the whole PPO configuration search, assumed constant. Log scale.

6.5 Choosing a strategy

Table 11 puts the test result of each strategy next to what it cost. Performance is shown twice: sampled, as a trained policy is evaluated, and greedy, as the game uses a net. For RL alone it lists both the single best run and the configuration search that found it, because a practitioner would not know in advance that 60-minute games and large batches are needed; the hybrids reused that configuration.

strategy	Charter sampled · greedy	progress sampled · greedy	simulated decisions	CPU	CPU co
Imitation (BC)	0/30	933	0.05 M	0.2 core-h	$0.0
RL only, one run (h60big)	27/30 · 14/30	2,693 · 2,370	10.1 M	98 core-h	$5.0
RL only, with the configuration search	27/30 · 14/30	2,693 · 2,370	27.7 M	248 core-h	$1
WSP only (v2)	22/30	2,191	≈ 0.30 M	≈ 2 core-h	$0.1
Hybrid, sparse (WSP + one RL run)	28/30 · 24/30	3,082 · 2,942	10.4 M	≈ 89 core-h	$4.5
Hybrid, all weights (WSP + one RL run)	29/30 · 17/30	3,153 · 2,656	10.4 M	≈ 90 core-h	$4.6

Table 11. Performance on the 30 test islands next to effort. Charter: islands that earn the Imperial Charter within 60 minutes; progress after 60 minutes. Sampled values use the best-checkpoint rule; greedy values use the final weights, the stronger greedy checkpoint (Table 7). CPU at $0.0513 per core-hour [5]; token costs are estimates [6]. The person who steered the work is not priced.

The choice follows from three questions. **Can the policy act stochastically?** If actions may be sampled, every strategy except imitation reaches the Charter on most islands, and the hybrid with free weights is best (29 of 30). If the policy must be deterministic, as in the game, RL alone falls to about half its Charters and the sparse hybrid is clearly best (24 of 30), followed by the hand-written net alone (22 of 30). **Must the policy stay readable?** Then WSP alone or the sparse hybrid: both keep every hand-written unit's meaning, and the hybrid's changes can be reviewed (Table 9). **What does a sample cost?** Here a simulated decision costs microseconds of CPU, and the hybrid's extra effort over WSP alone, one RL run of about 10 million decisions, costs about five dollars and an hour and a half. Where samples are expensive (a slow physical simulator, real hardware, people in the loop), WSP alone needs about thirty times fewer of them, and RL alone is the

most expensive option. A policy imitated from rules is the cheapest to make and the only one that never works here.

7 Discussion

Imitation is capped by what it imitates. The BC net reproduces the rule-based policy faithfully, including its blind spots: branches that policy never reached leave zero weights. No amount of data from it could teach cloth.

Writing the weights was the fastest way to a policy that plays the whole game, and it produced a network that is fully explained: 6 % of its weights are non-zero, every unit has a stated job, and the ablation confirms each job. It also overfits the islands its author watched: 14 of 14 Charters there, 22 of 30 on unseen islands. Hand-tuned thresholds are fitted to what the programmer saw, as stage 0 had already shown.

Its largest effect was outside the network. Six planner defects capped every policy, and they were found because a hand-written unit makes a specific promise ("this unit fires when no tavern covers 15 % of houses") that can be checked against a trace. A learned policy facing the same defects would simply have learned to avoid the actions that silently failed. After the fixes, even a uniformly random policy earns the Charter on almost half the islands.

Reinforcement learning finds what the programmer did not write down, given millions of decisions, a reward that encodes the goal and episodes long enough to reach it. It finds it as a stochastic policy; deployed greedily, the same net loses half its Charters.

The two combine well. The hand-written net gives reinforcement learning a start that already plays the whole game, and reinforcement learning corrects the programmer's priorities with evidence from millions of games. With a sparsity mask the result stays a program a person can read: the corrections in Table 9 agree with what the ablation and the development history had suggested, and they are small enough to review.

Writing weights is programming, with programming's failure modes. Several edits backfired in ways that only a full benchmark showed: a veto that never lifted, a gold threshold that held the treasury exactly below the price of a colony, an import that ate the colony's budget. These are interactions between rules, the same class of bug as in any rule system, and the one PPO fixed most decisively.

8 Scaling weight-space programming with a team of agents

The network here is small enough for one programmer. A natural question is whether the method scales by dividing the weights among several agents, each of which iteratively edits its own subset. We think it does, under conditions that this study makes concrete. We have not tested a multi-agent setup; this section is a hypothesis with a proposed test.

Why it can work. A hand-written network is modular by construction. Every unit of WSP v2 has its own inputs, its own actions and a stated purpose, and the units fall into nine groups (Table 4) that read like the modules of a program: economy, production chains, services, colonies, trade. An agent responsible for one group needs to read only that group's weights, which eases the context limit that the original weight-space programming experiments name as a constraint [1]. The ablation (Table 5) shows that most units can be tested in isolation: switching one off usually costs a small part of the progress and points to exactly the behaviour it owns.

Why it is not free. The modules are coupled through the output layer: priorities from all groups compete for the same decision. The most expensive bugs in this study were interactions between groups, each of which would have looked correct from inside its module. Importing liquor (trade) kept the treasury just below the price of a colony (colonies), so no colony was ever founded; a veto on houses (economy) waiting for a sheep farm (chains) stopped all growth when the farm found no site. A team therefore needs what software teams need. First, interfaces: fixed feature semantics and a priority band per group, so that one module cannot silently outbid another. Second, a shared integration test: the seeded benchmark, run after every change, with per-unit ablation as a regression check. Third, coordination: an agent or person who owns the cross-module priorities and reviews changes that touch them.

Where it stops. The argument holds for networks written to be modular. In large networks trained by gradient descent, units are typically polysemantic and features are stored in superposition, so a partition of the weights does not correspond to a partition of the functions, and edits by different agents would interfere. The weight-space edits reported for a 0.8-billion-parameter language model [1] accordingly targeted a single direction in activation space found by analysis, not a block of weights. For such models a team would first need interpretability tools that recover modules; for networks written from scratch, the modules exist by design.

A hand-written mixture of experts. The natural architecture for such a team is a mixture of experts (MoE) written by hand. In a trained MoE the experts do not split by subject: studies of trained MoE language models find little domain specialization, the experts couple through the shared residual stream, and the learned router depends on representations every expert helps to form. When the network is written instead of trained, the split is a design decision. Each expert can be given one subject, and the router can be written as readable conditions ("this state belongs to the colony expert because citizens are unlocked and no Kontor exists"). Every expert boundary is then a functional boundary, so the team divides along it: one agent per expert, with the router and the priority bands owned by the coordinator. WSP v2 is a miniature of this design. Its unit groups are the experts, the conditions inside each unit are the routing, and the action priorities are the interface between them. Two couplings remain even then. First, inputs that concern several subjects activate several experts, whose contributions must agree, as the liquor import (trade) and the colony budget (colonies) did not; hard routing, where exactly one expert decides, or a fixed priority band per expert keeps this small and visible. Second, all experts read the same features, so the meaning of a feature is an interface the coordinator must keep fixed. Hand-written

experts are realistic today for problems with engineered features, such as games, control and decision policies. Writing a language-model expert for a field such as medicine by hand is out of reach.

Coupling runs through resources, not weights. Measured in the weights of WSP v2, the modules are only loosely coupled: 3 of the 36 pairs of groups drive a common action (8 %), and 12 read a common input (33 %). The bugs that cost most in this study were invisible to that measure. Liquor import and colony founding share no weight; they share gold. Every module that spends a resource is coupled to every other one that does, so a team cannot rely on module-level tests: each change needs the full benchmark, and shared resources need to become explicit interfaces, for instance a gold budget per module. How many agents such a team needs, and how the cost of integration grows with the size of the network, this study cannot say; it had one agent and no subagents.

A test. Rebuild the advisor with one agent per group of Table 4, each given only its group's units, the feature definitions, its priority band and the benchmark, and a coordinating agent that merges changes after a full benchmark run. Compare the time to reach the Charter on all development islands and the held-out result against the single-programmer net, and count how many regressions the integration test catches that the module-level tests did not.

9 Limitations

The hand-written net was developed on the development islands; only the 30 test islands give an unbiased comparison, and 30 islands without confidence intervals leave differences of one or two Charters within noise. Each PPO configuration was trained once, with one seed, and only five configurations were tried. The checkpoint rule (best sampled validation score) was fixed before testing; the final weights, also reported, were better for greedy play. The programmer was an AI agent with full access to the source code, the simulator and single-game traces, steered by a person; a human programmer, or one without traces, may need far longer. Progress weights (1/3/6/10) are a choice. The planner fixes changed the environment during the study; all final numbers use the final environment, while the rows of Tables 1 and 2 each use the environment of their time. Token costs are estimates.

10 Conclusion

For a small policy over engineered features, writing the weights by hand was feasible, fast and fully interpretable, and it exposed defects that capped every method. Alone it overfits what its author saw and is beaten on unseen islands by reinforcement learning. Used as the starting point for reinforcement learning, it gives the best policy we found, at the same compute as RL from scratch, and with a sparsity mask the result remains a network whose every hand-written unit can still be read and whose changes by RL can be reviewed like a code diff. The game now ships that combination: the hand-written net fine-tuned with the sparsity mask. The study suggests a division of

labour: the programmer writes the structure and the knowledge that is easy to state, reinforcement learning tunes what is hard to state, and the network stays a program.

Reproduction

```
node dev/smoke.js                     # 116 headless assertions
node dev/handwritten-net-v2.js         # rebuild js/ai-weights.js (WSP v2)
node dev/explain-net.js                # Table 4 as text
node dev/train.js                      # BC net → dev/ai-weights-trained.json
RUN=h60big EP_STEPS=900 EPISODES=56 WORKERS=56 node dev/rl-train.js 200     #
RUN=wsprlsparse INIT=wsp-v2.json WARMUP=20 SPARSE=1 \
  EP_STEPS=900 EPISODES=56 WORKERS=56 node dev/rl-train.js 200
SET=test PPO=h60big,wsprl,wsprlsparse,… node dev/compare.js   # Table 7
node dev/ablate.js                     # Table 5 → dev/ablation.json
node paper/build.js                    # this paper
```

References

1. Weight-space programming. Blog post, rikiyat.github.io (accessed November 2026).
2. S. Ross, G. Gordon, D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning (DAgger). AISTATS 2011.
3. J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.
4. J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel. High-dimensional continuous control using generalized advantage estimation. ICLR 2016.
5. AWS EC2 on-demand pricing, c7a instances, us-east-1 (≈ $0.0513 per vCPU-hour); via calculator.holori.com, accessed November 2026.
6. Anthropic. Claude API pricing and model documentation (Claude Opus 5.5: $4 / $20 per million input / output tokens, cache reads $0.20; Claude Fable 5.1: $10 / $50; earlier: Claude 3 Opus $15 / $75, Claude Opus 4.5 $5 / $25).
7. Epoch AI. LLM inference prices have fallen rapidly but unequally across tasks. Data insight.
8. G. Appenzeller. Welcome to LLMflation: LLM inference cost is going down fast. Andreessen Horowitz, November 2024.
9. MIT FutureTech. The price of progress: algorithmic efficiency and the falling cost of AI inference. arXiv:2511.23455, 2025.